

Математические основы алгоритмов, осень 2020 г.
Лекция 6: Двоичные деревья поиска, AVL-деревья.
B-деревья*

Александр Охотин

3 апреля 2021 г.

Содержание

1	Двоичные деревья поиска	1
1.1	Структуры данных для представления множеств	1
1.2	Двоичные деревья поиска	1
1.3	AVL-деревья и их высота	3
1.4	Балансировка AVL-деревя	4
1.5	Исправление после вставки	5
1.6	Исправление после удаления	6
2	B-деревья	7
2.1	Вставка в B-дерево	8
2.2	Удаление в B-дереве	9
2.3	Понятие о красно-чёрных деревьях	10

1 Двоичные деревья поиска

1.1 Структуры данных для представления множеств

Структуры данных для представления *множества* различными способами, с разным временем выполнения операций (строго говоря, речь идёт о *мультимножестве*, поскольку может быть несколько элементов с одинаковыми значениями). Элементы могут быть любого вида, на них определено отношение порядка “ $\leq$ ”. Операции: найти элемент с данным значением; найти наименьший (наибольший) элемент; найти элемент, предшествующий по порядку данному (следующий за данным); вставить элемент; удалить элемент.

1.2 Двоичные деревья поиска

Двоичное дерево поиска (binary search tree, BST) — представление множества в виде дерева. Вершина содержит одно значение и три указателя: на предка (если это корень, то там NULL), на левое поддерево и на правое (если поддерева нет, то там NULL). При этом все вершины в её левом поддереве содержат не меньшие значения, а все вершины в правом поддереве — не бóльшие.

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.

Обход дерева: перечислить все элементы по порядку. Можно, например, рекурсией: сперва обойти левое поддерево, потом вывести значение в данной вершине, потом обойти правое. Но рекурсия не обязательна, достаточно помнить текущую вершину и откуда алгоритм в неё пришёл.

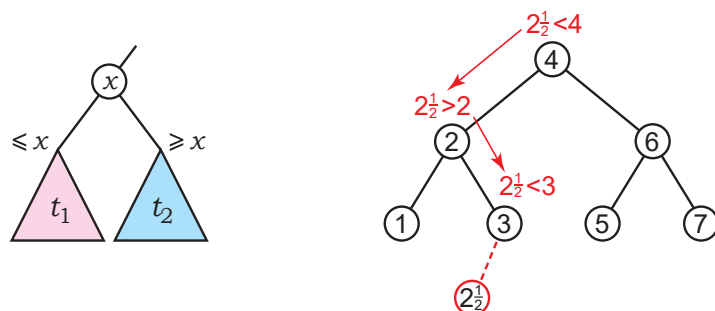


Рис. 1: Двоичное дерево поиска: (слева) условие для каждого поддерева; (справа) поиск элемента $2\frac{1}{2}$ с последующей вставкой.

Операции над деревом.

Поиск. Начиная с корня, сравнивать искомый элемент с текущим, и если они не равны, следовать в нужное поддерево.

Найти наименьший (наибольший элемент). Начиная с корня, переходить всё время к левому (правому) потомку. Никаких сравнений не требуется.

Найти следующий по порядку элемент. На входе — указатель на текущий элемент. Если у него есть правое поддерево, то это — наименьший элемент правого поддерева. Если же правого поддерева нет, то надо подниматься по дереву вверх, и как только текущее дерево окажется *левым* поддеревом чего-то, следующий элемент найден.

Предыдущий элемент — симметрично.

Добавление элемента. Сперва найти место, где он должен был бы быть — поиском вниз до листа. А далее сделать новый элемент левым или правым потомком этого листа, в зависимости от значения.

Удаление элемента x (дан указатель на него). Если у элемента x нет потомков, то его можно просто удалить. Если потомок один, то он занимает место x .

Если потомка два, то, как показано на рис. 2, находится элемент y , следующий по порядку за x (он в его правом поддереве). Он и занимает место удаляемого элемента. Если у y есть правый потомок, то тот в свою очередь занимает освободившееся место, где был y .

Сложность каждой операции — не более чем высота дерева. Если дерево сбалансировано — т.е., все пути примерно одинаковой длины — то все операции выполняются за логарифмическое время.

Как обеспечить сбалансированность дерева? Даже если оно сбалансировано изначально, то операции добавления и удаления элементов его разбалансируют. Поэтому необходимо тем или иным образом усложнить структуру данных, ввести какие-то дополнительные ограничения на неё, чтобы после выполнения каждой операции дерево автоматически балансировалось бы.

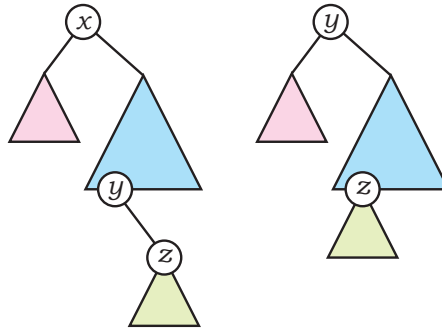


Рис. 2: Двоичное дерево поиска: удаление вершины x , у которой есть оба потомка.

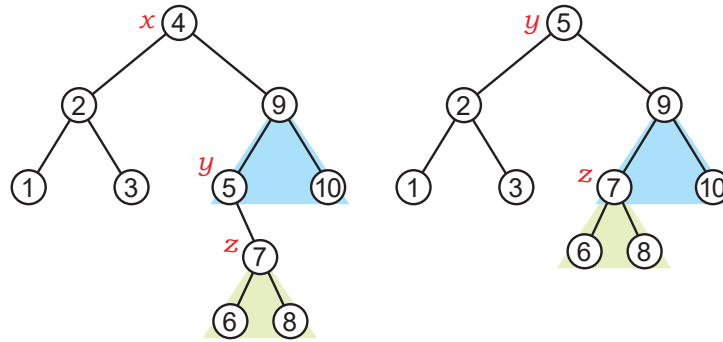


Рис. 3: Удаление в двоичном дереве поиска: удаляется $x = 4$.

Требование, чтобы все поддеревья содержали одинаковое число элементов, или же отличающееся не более чем на один — бессмысленно жёсткое, оно делает балансировку трудоёмкой. Дереву нужно дышать! Нужно позволить ему быть местами немного разбалансированным, чтобы при добавлении элементов в малонаселённые места или при удалении из густонаселённых мест ничего балансировать не приходилось бы. Есть несколько разновидностей таких деревьев.

1.3 AVL-деревья и их высота

AVL-деревья (Адельсон-Вельский и Ландис [1962]; AVL trees): усложнённая разновидность двоичных деревьев поиска.

«Почти сбалансированное» двоичное дерево, в котором высота поддеревьев-потомков одной вершины отличается не более чем на 1. Этого ограничения достаточно, чтобы дерево имело логарифмическую высоту. «Дышат» за счёт трёх возможных разностей высоты.

Дополнительно в каждой вершине хранится высота её поддерева — достаточно даже не высоты, а разности между высотой правого и левого поддеревьев.

Лемма 1. AVL-дерево высоты h содержит не менее чем $F_{h+3} - 1$ вершин, где F_n — n -е число Фибоначчи.

Леонардо Пизанский (Фибоначчи) (ок. 1175–ок. 1250): последовательность $F(n)$. В первый месяц была одна пара кроликов. Каждый месяц каждая пара возраста не менее двух месяцев производит на свет новую пару. Кролики не умирают. Сколько будет пар на n -м месяце? $F_1 = 1$, $F_2 = 1$, $F_n = F_{n-1} + F_{n-2}$ — то есть, все, кто были в прошлом месяце, есть

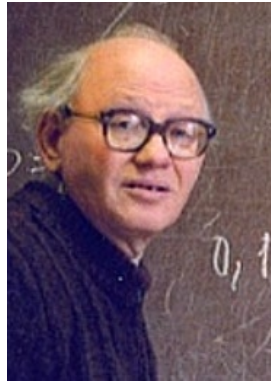


Рис. 4: Георгий Адельсон-Вельский (1922–2014), Евгений Ландис (1921–1997).

и сейчас, и все, кто были два месяца назад, производят потомство. В качестве начальных условий удобнее положить $F_0 = 0$, $F_1 = 1$.

Доказательство леммы 1. Индукция по h .

Базовый случай: $h = 0$, это одинокая вершина, и $F_{0+3} - 1 = 1$.

Переход: если дерево имеет высоту h , то один из его потомков имеет высоту $h - 1$, и потому содержит не менее чем $F_{h+2} - 1$ вершину, а другой — высоту не менее чем $h - 2$, и, стало быть, не менее чем $F_{h+1} - 1$ вершину. Всего, с учётом корня, $1 + F_{h+2} - 1 + F_{h+1} - 1 = F_{h+3} - 1$ вершин. $\square$

А сколько это?

Утверждение 1. $F_n = \frac{1}{\sqrt{5}}(\varphi^n - \psi^n)$, где $\varphi = \frac{1+\sqrt{5}}{2}$ (золотое сечение) и $\psi = \frac{1-\sqrt{5}}{2}$.

Отсюда $F_n = \frac{1}{\sqrt{5}}\varphi^n - o(1)$. Приближённые значения: $\varphi \approx 1.62$ и $\psi \approx -0.62$.

Лемма 2. Высота AVL-дерева с n вершинами не превосходит $\log_\varphi n$.

Поэтому поиск в AVL-дереве занимает логарифмическое время.

При вставке элемент вставляется, как в обычное дерево поиска. Однако при этом дерево может разбалансироваться, то есть, у некоторых вершин высота поддеревьев станет различаться на 2 (но не более чем на 2). Поэтому после вставки всегда проводится *исправление после вставки*, чтобы дерево осталось правильным AVL-деревом.

Элемент удаляется, как из обычного дерева поиска, с последующим *исправлением после удаления*.

1.4 Балансировка AVL-дерева

Пусть разность высоты поддеревьев-братьев — не более чем 2, и все такие разбалансированные вершины находятся на одном пути из корня. Тогда находится наименьшее разбалансированное поддерево (где разность равна двум), и разбалансировка проталкивается вверх по дереву. Если высота этого поддерева — h , то у него два поддерева высоты $h - 1$ и $h - 3$. У поддерева высоты $h - 1$ — свои поддерева: одно высоты $h - 2$, другое высоты $h - 3$ или $h - 2$.

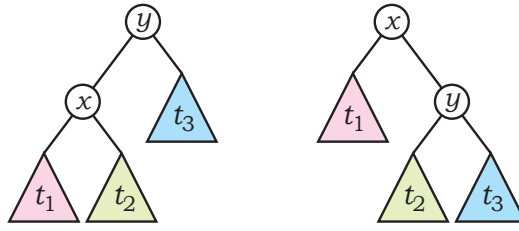


Рис. 5: Вращение между вершинами x и y в AVL-деревьях.

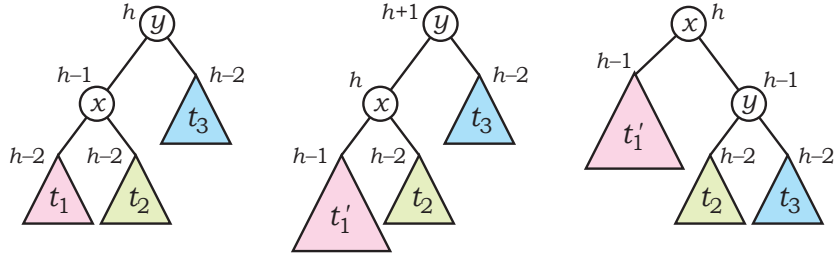


Рис. 6: Исправление после вставки, случай «левый-левый», одно вращение: (слева) исходное дерево; (посередине) дерево после вставки в t_1 ; (справа) дерево после вращения.

Четыре случая, в зависимости от того, через каких потомков (правых или левых) идёт путь в поддерево высоты $h - 2$: левый-левый, левый-правый, правый-левый и правый-правый.

Основная элементарная операция: *вращение* (или *поворот*). Пусть в дереве поиска есть две соседние вершины x, y , где x — левый потомок y , у x поддерева t_1 и t_2 , а правый потомок y — t_3 ; схематично: $(t_1 x t_2) y t_3$. Тогда эту структуру внутри дерева можно изменить, поставив x наверх и получив $t_1 x (t_2 y t_3)$ — дерево останется правильным деревом поиска. Это — *вращение между x и y* . Точно так же можно вращать и в обратном направлении.

Ниже приведены подробности применения вращения для балансировки дерева после операций вставки и удаления.

1.5 Исправление после вставки

Все разбалансированные вершины находятся на пути из корня во вставленный лист. Рассматривается самая нижняя из разбалансированных вершин, y . Пусть h — высота вершины y до операции вставки, пусть высота её потомков до вставки — $h - 1$ и $h - 2$, и пусть вставка производится в потомке, имеющем высоту $h - 1$; на рис. 6(левом) этот потомок обозначается через x . Потомки x имеют одну и ту же высоту $h - 2$ (действительно, имей они разную высоту, после вставки в более низкое поддерево разбалансировки вообще не возникло бы, а после вставки в более высокое поддерево разбалансировка возникла бы уже в вершине x).

После вставки вершина x получит высоту h , а вершина y — высоту $h + 1$, как изображено на рис. 6(среднем). На этом рисунке показан случай, когда вставка производится в левом потомке левого потомка вершины y — её «левом-левом» потомке. Бросается в глаза, что поддерево t'_1 выше двух других, однако прицеплено снизу — и хочется переставить эти поддерева так, чтобы t'_1 оказалось прицеплено на уровень выше. Для исправления достаточно одного вращения между x и y : на рис. 6(правом) представлено получившееся исправленное поддерево, которое опять имеет высоту h . Стало быть, разбалансировка исправлена одним этим вращением.

Случай вставки в правом-правом потомке самой нижней разбалансированной вершины симметричен.

Пусть вершина вставлена в левом-правом потомке, как показано на рис. 7(сверху). Пусть

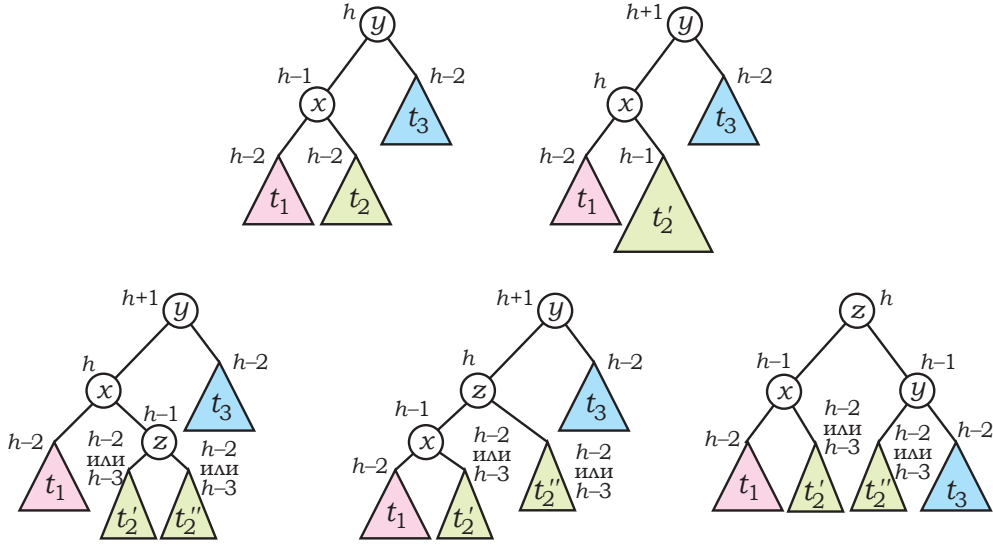


Рис. 7: Исправление после вставки: случай «левый-правый». Сверху: слева исходное дерево до вставки, справа — после вставки. Снизу: слева структура левого-правого потомка z , посередине — после первого вращения, справа — после второго вращения.

z — этот потомок. Тогда после вставки z имеет высоту $h - 1$, а высота каждого из его потомков — или $h - 2$, или $h - 3$. Этот случай изображён на рис. 7(снизу слева). Тогда сперва производится вращение между x и z , получается дерево, как на рис. 7(снизу посередине), а потом делается ещё одно вращение между z и y . Итоговое поддерево, приведённое на рис. 7(снизу справа), сбалансировано и имеет высоту h .

Случай вставки в правом-левом потомке симметричен.

Во всех случаях высота исправленной вершины становится такой же, как была до вставки, и потому разбалансированность не распространяется выше по дереву.

1.6 Исправление после удаления

Вершина удаляется из АВЛ-дерева, как из обычного дерева поиска. После удаления рассматривается самая нижняя разбалансированная вершина: если её высота — h , то одно из её поддеревьев имеет высоту $h - 1$, а другое (то, где проведено удаление) — $h - 3$. В зависимости от структуры поддерева высоты $h - 1$, есть три случая исправления. Если поддерево состоит из двух поддеревьев высоты $h - 2$, как на рис. 8, то исправление ограничивается одним вращением. Высота дерева остаётся равной h , поэтому сверху ничего не меняется.

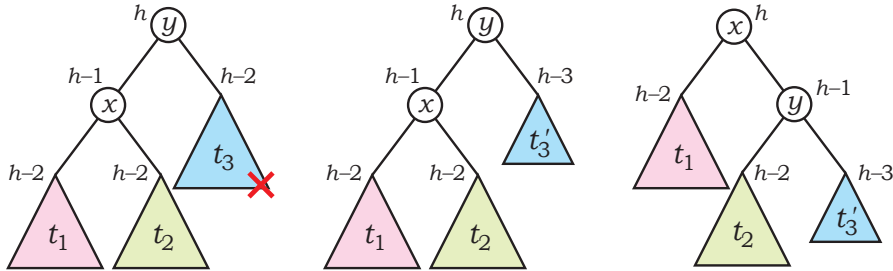


Рис. 8: Исправление после удаления, случай «слева одинаковые», одно вращение: (слева) исходное дерево; (посередине) дерево после удаления из t_3 ; (справа) дерево после вращения.

Пусть поддерево высоты $h - 1$ — левое, и пусть его левое поддерево выше правого. То-

гда высота двух поддеревьев левого поддерева — $h - 2$ и $h - 3$, как изображено на рис. 9. После одного вращения условия АВЛ-дерева выполнены, однако высота исходной вершины уменьшилась с h до $h - 1$. В этом случае условия АВЛ-дерева могут нарушиться для каких-то вершин, расположенных выше. Поэтому алгоритм продолжает исправление после удаления, поднявшись на уровень выше. В крайнем случае так придётся дойти до самого корня, уменьшение высоты которого уже не нарушит никаких условий.

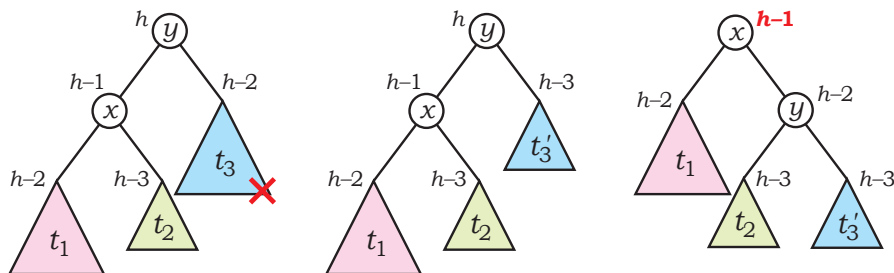


Рис. 9: Исправление после удаления, случай «слева больше левое», одно вращение: (слева) исходное дерево; (посередине) дерево после удаления из t_3 ; (справа) дерево после вращения, его высота уменьшилась.

Наконец, если поддерево высоты $h - 1$ — левое, и его левое поддерево ниже правого, то высота правого-левого поддерева — $h - 2$, и оно состоит из двух поддеревьев, каждое из которых имеет высоту $h - 3$ или $h - 4$. Тогда, как показано на рис. 10, четыре указанных поддерева выравниваются на один уровень, что можно сделать двумя операциями вращения. В итоге высота исходной вершине уменьшится до $h - 1$ и исправление после удаления продолжится на уровень выше.

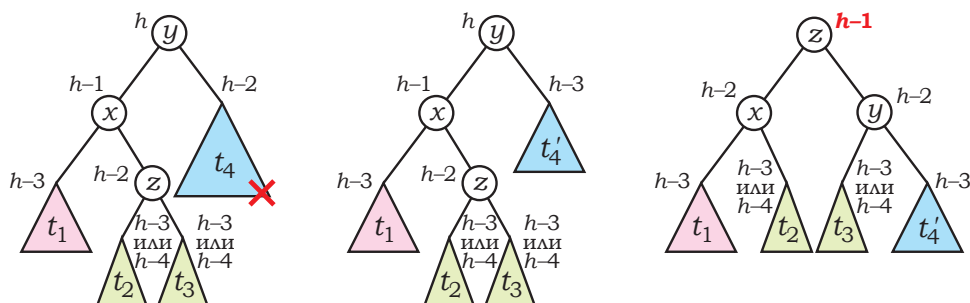


Рис. 10: Исправление после удаления в АВЛ-дереве, случай «слева больше правое», два вращения: (слева) исходное дерево; (посередине) дерево после удаления из t_4 ; (справа) дерево после двух вращений, его высота уменьшилась.

2 В-деревья

Двоичные деревья поиска рассчитаны на хранение в оперативной памяти компьютера, позволяющей за одну операцию обратиться не более чем к нескольким байтам. Каждая вершина дерева может быть обработана за несколько таких операций, и такое использование оперативной памяти оптимально.

Для хранения деревьев во внешней, медленной памяти (такой, как жёсткий диск) структуру данных необходимо адаптировать. Главная особенность внешней памяти в том, что за одну операцию читается или записывается блок данных размером в несколько килобайт — например, сектор на жёстком диске. Поэтому для доступа к одной вершине двоичного

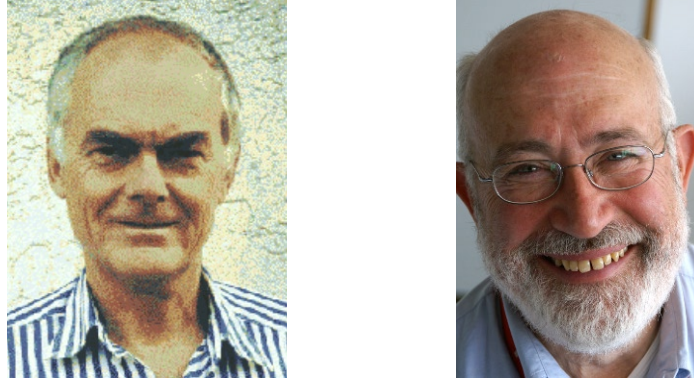


Рис. 11: Рудольф Байер (род. 1939), Эдвард Маккрайт.

дерева пришлось бы работать с целым блоком, и поиск в дереве с n вершинами потребовал бы порядка $\log_2 n$ операций с блоками. Это неоптимально.

Предложенные Байером и Маккрайтом [1972] *В-деревья* — это адаптация деревьев поиска для хранения во внешней памяти. Главная мысль — использовать вершины большой степени — с тем, чтобы каждая вершина занимала один блок, а высота дерева уменьшилась бы. Например, если все вершины имеют степень 1000, то высота дерева с миллиардом вершин будет равна всего лишь трём (а не тридцати, как у двоичного дерева), и поиск нужного листа потребует прочитать лишь 4 блока.

Пусть вершины в двоичном дереве — это «2-вершины», поскольку у каждой из них 2 потомка и 1 значение, по которому эти потомки разделяются. У m -вершины — m потомков (деревья $t_1, \dots, t_m$ — возможно, пустые), и в ней находится $m - 1$ значение: $x_1, \dots, x_{m-1}$, где $x_1 \leq \dots \leq x_{m-1}$. Все значения в каждом поддереве t_i больше или равны x_i и меньше или равны x_{i+1} , как показано на рис. 12.

В В-дереве могут одновременно содержаться вершины различных степеней: выбирается некоторое число $k \geq 2$, после чего корень может иметь степень от 0 до $2k$, а все остальные вершины — любые степени от k до $2k$. При этом дерево сбалансировано: *длины всех путей равны*.

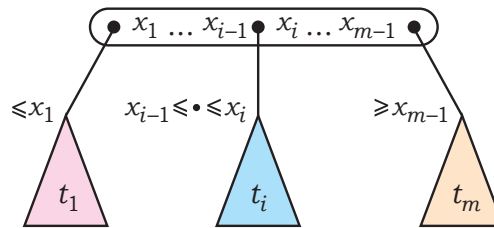


Рис. 12: m -вершина в В-дереве

При поиске элемента x в В-дереве, на каждом шаге рассматривается некоторая m -вершина, в которой размещены значения $x_1, \dots, x_{m-1}$. В этом массиве запускается двоичный поиск элемента x . Если x находится в массиве, то поиск в дереве на этом завершён, а если нет, то двоичный поиск указывает на поддерево t_i , для которого верно $x_{i-1} < x < x_i$, и потому элемент x может находиться только в нём. Поиск продолжается в поддереве t_i .

2.1 Вставка в В-дереве

В АВЛ-дереве вставка и удаление начинается с поиска данного элемента, после чего исправляется возможная разбалансировка. В худшем случае придётся пройти путь от корня

к листу, а потом обратно от листа к корню.

В В-дереве вставка и удаление делаются иначе: исправление потенциальной разбалансировки начинается уже на этапе поиска удаляемого элемента в дереве.

Пусть вставка или удаление производятся в листе, и это m -вершина. Тогда, если этот лист заполнен не до конца ($m < 2k$), то в нём найдётся место для дополнительного значения, а если он заполнен не минимально ($m > k$), то из него можно удалить любое из его значений.

При вставке, спускаясь вниз по дереву, нужно разделять каждую очередную встреченную «полную» $2k$ -вершину на две k -вершины — за счёт её сестёр или родительницы. При этом одно лишнее значение выталкивается на один уровень выше, как на рис. 13. Поскольку на уровне выше не может быть $2k$ -вершины (ведь алгоритм только что оттуда спустился), в ней есть место, в которое можно вытолкнуть лишнее значение. В итоге найденный лист тоже окажется степени не более чем $2k - 1$ — то есть, в нём будет не более чем $2k - 1$ пустых указателей на несуществующих потомков, и между ними не более чем $2k - 2$ значений; следовательно, в этом листе найдётся, куда вставить новое значение.

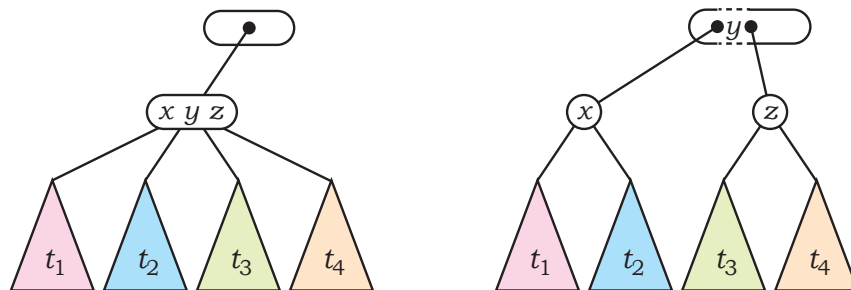


Рис. 13: Разделение вершины при вставке в В-дереве, для $k = 2$: (слева) переполненная 4-вершина; (справа) выталкивание переполнения вверх.

Что делать, если при вставке окажется, что степень корня — $2k$? Здесь важно, что степень корня не ограничена снизу. Тогда корень точно так же разделяется на две k -вершины, лишнее значение так же выталкивается на уровень выше, где появляется новый корень степени 2, с одним значением. Высота дерева увеличивается только в этом случае.

2.2 Удаление в В-дереве

При удалении, спускаясь вниз, нужно точно так же увеличивать каждую встреченную k -вершину за счёт её сестёр. Для всякой встреченной k -вершины уже обеспечено, что её родительница — не менее чем $(k + 1)$ -вершина. Значит, у текущей k -вершины есть не менее k сестёр.

- Если одна из соседних сестёр — не менее чем $(k + 1)$ -вершина, то недостающее поддерево перегоняется из неё, как показано на рис. 14.
- Если же соседняя сестра — k -вершина, то она объединяется с текущей k -вершиной в одну $2k$ -вершину — это в точности обратная операция к разделению вершины, изображённой на рис. 13. При этом у родительницы станет на одно поддерево меньше, но она это переживёт, поскольку она не k -вершина.

Когда же наконец находится вершина, содержащая удаляемое значение, если это лист, то значение просто удаляется (он же степени хотя бы $k + 1$!). Если же нужно удалить элемент, находящийся во внутренней вершине, то запоминается указатель на эту вершину, после чего делается следующее.

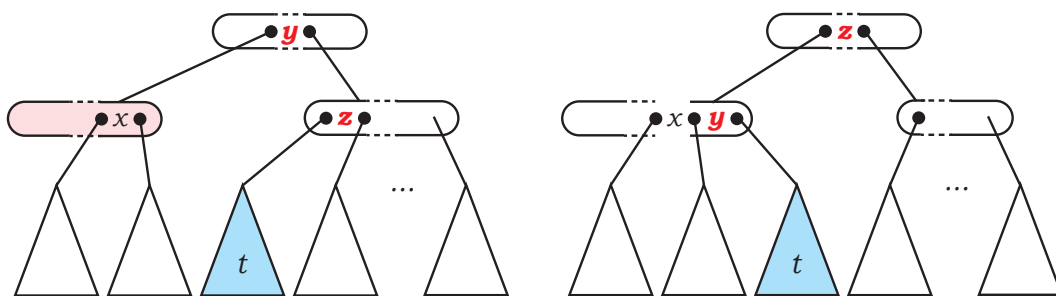


Рис. 14: Заимствование поддерева у соседней сестры при удалении в В-дереве: (слева) малоимущая k -вершина, выделенная красным; (справа) поддерево заимствовано у сестры справа.

- Если в предшествующем значению поддерева, в его корне есть хотя бы $k + 1$ значение, то в поддерева находится значение-предшественник, и процедура стирания продолжается уже для него (а само оно замещает стираемый элемент).
- Аналогично в следующем поддерева.
- Если же у соседних поддеревьев по k значений, то они объединяются в одно с $2k$ значениями.

Корень нужно рассмотреть особо. Поскольку его степень не ограничена снизу, сам по себе он не нуждается в исправлении. Однако если корень — 2-вершина, а оба его потомка — k -вершины, то исправление этих k -вершин приведёт к полному опустошению корня. В этом случае все три вершины объединяются в одну $2k$ -вершину, а высота дерева уменьшается на единицу.

2.3 Понятие о красно-чёрных деревьях

В-дерево для $k = 2$ называется *2-3-4-деревом*, и такое дерево удобно хранить в оперативной памяти. Его высота незначительно меньше, чем у АВЛ-деревя, однако оно требует большого объёма кода для своей реализации.

Красно-чёрное дерево — это представление 2-3-4-деревя в виде двоичного деревя, в котором каждая вершина 2-3-4-деревя представлена в виде одной или нескольких связанных между собою двоичных вершин, каждая из которых покрашена в один и двух цветов, красный или чёрный. Каждая 2-вершина остаётся собой и считается чёрной. Каждая 3-вершина разбивается на две двоичных: чёрную вершину — корень поддерева, и красную вершину — правого или левого потомка чёрной; к этой паре вершин, так же, как и к исходной 3-вершине, присоединяются три поддерева. Наконец, каждая 4-вершина разбивается на три двоичных — чёрный корень поддерева и её два потомка — красные вершины, к которым присоединены четыре поддерева.

На красно-чёрное дерево налагается следующее ограничение: две красных вершины не могут быть соединены ребром. От В-деревьев также унаследовано свойство равенства длин всех путей — в случае красно-чёрных деревьев требуется, чтобы количество чёрных вершин на каждом пути из корня к листу было одинаковым. Благодаря этим двум условиям, красно-чёрные деревья взаимно однозначно соответствуют 2-3-4-деревьям.

Использование красно-чёрных деревьев вместо 2-3-4-деревьев позволяет заменить сложные операции для В-деревьев несколькими более простыми операциями над красно-чёрными деревьями. Поэтому красно-чёрные деревья существенно удобнее программировать, и они успешно используются на практике.

Список литературы

- [1962] Г. М. Адельсон-Вельский, Е. М. Ландис, “Один алгоритм организации информации”, *Доклады Академии наук СССР*, 146:2 (1962), 263–266.
- [1972] R. Bayer, E. M. McCreight, “Organization and maintenance of large ordered indexes”, *Acta Informatica*, 1:3 (1972), 173–189.
- [1978] L. J. Guibas, R. Sedgwick, “A dichromatic framework for balanced trees”, *FOCS 1978*, 8–21.